

INTERNSHIP PROJECT REPORT

DocuQuery RAG

Grounded Document Question-Answering System using FastAPI, FAISS & Google Gemini

Submitted in partial fulfillment of the requirements for the
Internship Program in AI Engineering

Organization	SHADOWFOX
Internship Domain	AI Engineer
Duration	August 1, 2026 – September 2, 2026 (Approx. 1 Month)
College	PKIET
Degree Program	B.Tech – Information Technology (IT)
Year of Study	3rd Year

Submitted by:
ILAMARAN R

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Technologies Used
6. System Requirements
7. System Architecture & Workflow
8. Project Features and Modules
9. Implementation / Methodology
10. User Interface & Screenshots
11. Testing and Results
12. Advantages
13. Limitations
14. Future Enhancements
15. Conclusion
16. References

1. Abstract

This report documents the design and development of **DocuQuery RAG**, a grounded document Question-Answering (QA) system built during a one-month AI Engineering internship at **SHADOWFOX**. The system implements a Retrieval-Augmented Generation (RAG) pipeline that allows a user to upload PDF or TXT documents, automatically extracts and chunks their textual content, converts the chunks into vector embeddings using the **Google Gemini Embedding API**, and stores them in an in-memory **FAISS** vector index. When a user asks a question, the system retrieves the most semantically relevant chunks from the index and passes them, along with a strict grounding prompt, to the **Gemini LLM (gemini-3.5-flash)** to generate an answer that is based exclusively on the retrieved document content — rather than the model's general world knowledge. The backend is implemented using **FastAPI**, exposes a documented REST API (via Swagger/OpenAPI), and is paired with a custom glassmorphic web front-end for document upload and interactive querying. The project demonstrates practical skills in backend API design, vector databases, LLM integration, prompt engineering, and full-stack application development.

2. Introduction

Retrieval-Augmented Generation (RAG) is a widely adopted architecture in modern AI engineering that combines the reasoning and language-generation capabilities of Large Language Models (LLMs) with an external, verifiable knowledge source. Instead of relying purely on an LLM's internal training data — which can be outdated, incomplete, or prone to hallucination — a RAG system first retrieves relevant information from a dedicated document store and then conditions the LLM's response on that retrieved evidence.

As part of the AI Engineer internship at SHADOWFOX, this project was undertaken to design and build a working, end-to-end RAG backend and companion web interface named **DocuQuery RAG**. The application allows users to ingest their own PDF or plain-text documents and then ask natural-language questions about the content of those documents, receiving answers that are strictly grounded in — and traceable back to — the source material, complete with the originating file name, page number, and a similarity/relevance score for each retrieved chunk.

The project involved working across the full stack of a modern AI application: document parsing, text chunking, vector embeddings, similarity search, LLM prompt design, REST API development, and front-end integration — giving practical, applied exposure to the core building blocks used in production-grade AI systems today

3. Problem Statement

General-purpose LLMs answer questions using only the knowledge they were trained on. This creates two key problems when a user wants information about a specific, private, or recently created document (such as a report, research paper, or internal manual):

- The LLM has no access to the document's actual content, so it cannot answer questions specific to it.
- If asked anyway, the LLM may 'hallucinate' — producing plausible-sounding but factually incorrect answers that are not grounded in the real source document, with no way for the user to verify where the answer came from.

DocuQuery RAG addresses this problem by building a pipeline that retrieves the most relevant sections of an uploaded document for a given question and forces the LLM to answer strictly using that retrieved context, explicitly refusing to answer when the context is insufficient. This ensures answers are accurate, traceable to a specific document and page, and free of ungrounded speculation.

4. Objectives

- To design and implement a REST API backend using FastAPI that can accept PDF/TXT document uploads.
- To extract text from documents on a page-by-page basis and split it into overlapping, semantically coherent chunks suitable for embedding.
- To generate vector embeddings for each chunk using the Google Gemini Embedding API and store them in a FAISS vector index for efficient similarity search.
- To implement a grounded Question-Answering pipeline that retrieves the top-k most relevant chunks for a user's question and uses a strict prompt template to force the Gemini LLM to answer only from the retrieved context.
- To expose retrieved source metadata (document name, page number, relevance/distance score) alongside every answer for transparency and verifiability.
- To build a simple, responsive web front-end that allows drag-and-drop document upload and conversational querying without requiring the user to interact with the API directly.
- To implement robust error handling, including graceful handling of unsupported file types, empty files, and Gemini API rate-limit (429) errors with automatic retry and exponential backoff.
- To validate the backend using an automated unit and integration test suite.

5. Technologies Used

The following technologies, frameworks, and libraries — verified directly from the project's `requirements.txt` and source code — were used to build DocuQuery RAG:

Category	Technology / Library	Purpose in Project
Backend Framework	FastAPI	REST API routing, request/response validation, CORS, automatic OpenAPI/Swagger docs
ASGI Server	Uvicorn	Running and serving the FastAPI application (with <code>--reload</code> for development)
Language	Python 3	Core implementation language for the entire backend
Document Parsing	PyMuPDF (fitz)	Extracting page-wise text content from uploaded PDF files
Text Chunking	LangChain (langchain-text-splitters)	<code>RecursiveCharacterTextSplitter</code> for splitting extracted text into overlapping chunks
Document Abstraction	LangChain Core (Document)	Standardized in-memory representation of text chunks with metadata
Embeddings	Google Gemini Embedding API (langchain-google-genai)	Converting text chunks into vector embeddings (model: <code>models/gemini-embedding-001</code>)
LLM	Google Gemini (gemini-3.5-flash)	Generating grounded natural-language answers from retrieved context
Vector Database	FAISS (faiss-cpu)	In-memory similarity search index for storing and querying document embeddings
Data Validation	Pydantic / pydantic-settings	Request/response schema validation and typed configuration management
Config Management	python-dotenv	Loading environment variables (API keys, model names) from a <code>.env</code> file
File Uploads	python-multipart	Handling multipart/form-data file uploads in FastAPI
Frontend	HTML5, CSS3 (custom glassmorphism design), JavaScript (Fetch API)	Single-page web client for document upload and question answering
Testing	pytest, FastAPI TestClient	Unit and API-level automated testing
API Documentation	Swagger UI / OpenAPI (auto-generated by FastAPI)	Interactive API documentation at <code>/docs</code>

6. System Requirements

6.1 Software Requirements

- Python 3.10 or higher
- pip package manager (with packages listed in requirements.txt)
- A valid Google Gemini API Key (obtained from Google AI Studio)
- A modern web browser (Chrome, Edge, or Firefox) to access the web UI
- Operating System: Windows / Linux / macOS (verified running on Windows via PowerShell, as seen in the development terminal)

6.2 Hardware Requirements

- A standard development machine capable of running Python and a local ASGI server (Uvicorn)
- Internet connectivity, required for calling the Google Gemini embedding and LLM APIs
- Minimum 4 GB RAM recommended (FAISS index is held in memory for the session)

6.3 Key Python Dependencies

fastapi, uvicorn, pymupdf, langchain, langchain-community, langchain-core, langchain-text-splitters, langchain-google-genai, faiss-cpu, python-dotenv, pydantic, pydantic-settings, python-multipart (as pinned in requirements.txt).

7. System Architecture & Workflow

DocuQuery RAG follows a modular backend architecture in which each Python module has a single, well-defined responsibility. The end-to-end data flow is described below.

7.1 Module Overview

Module	Responsibility
main.py	Defines FastAPI app, CORS middleware, request/response Pydantic models, and all REST endpoints (/health, /upload, /query, /clear, /)
config.py	Loads and centralizes configuration (API key, embedding/LLM model names, chunk size, chunk overlap, top-k) from environment variables using pydantic-settings
document_processor.py	Extracts raw page-wise text from PDF (PyMuPDF) or TXT files, and splits it into overlapping chunks using LangChain's RecursiveCharacterTextSplitter
vector_store.py	Thread-safe VectorStoreManager class wrapping a FAISS index; handles embedding generation, similarity search, index statistics, and retry/backoff on API rate limits
rag_pipeline.py	RAGPipeline class that retrieves top-k relevant chunks, builds a strict grounding prompt, and calls the Gemini LLM to generate the final answer
static/ (index.html, styles.css, app.js)	Single-page glassmorphic web client for uploading documents, viewing ingestion stats, and asking questions with live results
test_rag.py	Automated test suite validating configuration, document extraction, chunking, and API endpoint behaviour

7.2 End-to-End Workflow

Document Ingestion Flow:

1. User uploads a .pdf or .txt file via the web UI (drag-and-drop or Browse Files) or directly via the POST /upload API endpoint.
2. main.py validates the file extension and reads its bytes.
3. document_processor.py extracts text (page-by-page for PDFs, via PyMuPDF) and splits it into overlapping chunks (default: chunk size 1000 characters, overlap 200 characters).
4. vector_store.py sends each chunk to the Gemini Embedding API (models/gemini-embedding-001) to generate vector embeddings, which are added to the in-memory FAISS index.
5. The API responds with the number of chunks added and the running total, which the UI displays under 'Ingested Documents' and 'Vector Chunks'.

Query / Question-Answering Flow:

1. User types a natural-language question in the 'Ask Questions' panel and clicks 'Ask Gemini' (or calls POST /query directly).
2. rag_pipeline.py calls vector_store_manager.similarity_search() to retrieve the top-k most relevant chunks (default k = 4) along with their similarity distance scores.
3. The retrieved chunks are formatted into a context block and inserted into a strict grounding prompt template that instructs the LLM to answer only from the given context.
4. The prompt is sent to the Gemini LLM (gemini-3.5-flash) via ChatGoogleGenerativeAI.

5. The generated answer, along with the retrieved sources (document name, page number, and relevance score for each chunk), is returned to the client and rendered in the 'Grounded AI Response' and 'Retrieved Context Chunks' sections of the UI.

8. Project Features and Modules

Multi-format Document Ingestion: Supports both PDF (parsed page-by-page using PyMuPDF) and plain TXT files (with UTF-8 and Latin-1 fallback decoding), with validation for unsupported file types and empty files.

Intelligent Text Chunking: Uses LangChain's RecursiveCharacterTextSplitter with a configurable chunk size (default 1000 characters) and overlap (default 200 characters) to preserve context across chunk boundaries.

Semantic Vector Search (FAISS): Maintains a thread-safe, in-memory FAISS vector index that supports incremental addition of new document chunks and top-k similarity search with distance scoring.

Grounded Question Answering: A strict RAG prompt template instructs the Gemini LLM to answer solely from retrieved context and explicitly state when the context is insufficient to answer, minimizing hallucination.

Source Transparency: Every answer is accompanied by the exact retrieved chunks, each tagged with its source filename, page number, and relevance/distance score, enabling users to verify the answer against the original document.

Rate-Limit Resilient API Calls: Both the embedding and LLM calls implement retry logic with exponential backoff on Gemini API 429 (rate limit / quota exceeded) errors.

REST API with Auto-Generated Docs: FastAPI automatically generates interactive Swagger UI documentation (at /docs) covering all endpoints, request schemas, and example responses.

Vector Index Management: A dedicated /clear endpoint and 'Clear Vector Index' UI button allow the in-memory index and document statistics to be reset at any time.

Health/Status Monitoring: The /health endpoint reports system status, whether the Gemini API key is configured, the active embedding/LLM model names, and current vector store statistics.

Responsive Glassmorphic Web UI: A custom-designed single-page front-end (HTML/CSS/JS) featuring a drag-and-drop upload zone, live ingestion statistics, a question input panel, and a formatted, Markdown-aware answer display.

Automated Test Suite: test_rag.py uses pytest and FastAPI's TestClient to verify configuration defaults, text extraction, chunking behaviour, and core API endpoint

9. Implementation / Methodology

9.1 Configuration Management

All tunable parameters — the Gemini API key, embedding model name (models/gemini-embedding-001), LLM model name (gemini-3.5-flash), chunk size, chunk overlap, and top_k retrieval count — are centralized in a Pydantic Settings class (config.py) and loaded from a .env file via python-dotenv. This keeps secrets out of source code and allows easy tuning without modifying application logic.

9.2 Document Processing Pipeline

The document_processor.py module defines two extraction functions — one for PDFs (using PyMuPDF's fitz.open() to iterate over each page and extract raw text into a LangChain Document with source and page metadata) and one for TXT files (with automatic UTF-8/Latin-1 fallback). A custom DocumentProcessingError exception is raised for corrupt files, unsupported extensions, or empty content. The extracted per-page Documents are then passed through a RecursiveCharacterTextSplitter to produce final overlapping chunks ready for embedding.

9.3 Vector Store & Embedding Layer

The VectorStoreManager class in vector_store.py wraps a LangChain FAISS vector store instance behind a threading.Lock to make it safe for concurrent API requests. On first document upload, FAISS.from_documents() initializes the index using GoogleGenerativeAIEmbeddings; subsequent uploads call add_documents() to append new vectors. The class also exposes similarity_search(), which performs similarity_search_with_score() to return both the matched chunks and their distance scores, and get_stats(), which reports total chunk count, document count, and per-source chunk breakdown for the /health endpoint and UI. Both add_documents() and similarity_search() implement retry logic with exponential backoff (wait_seconds = (attempt + 1) * 3) specifically for Gemini API 429 rate-limit errors.

9.4 Grounded RAG Prompting

The core of the grounding strategy lies in the RAG_PROMPT_TEMPLATE defined in rag_pipeline.py. The prompt explicitly instructs the LLM to rely only and exclusively on the provided document context, to avoid outside knowledge or extrapolation, and to respond with a fixed refusal message when the context is insufficient. The retrieved chunks are formatted into numbered snippets tagged with their source file and page number before being interpolated into the prompt, giving the LLM explicit citations to work from.

Example prompt structure used in the project:

```
CRITICAL RULES:
1. Answer the question relying ONLY and EXCLUSIVELY on the provided
Document Context below.
2. Do NOT use outside knowledge, prior assumptions, or extrapolate
beyond the explicit facts in the context.
3. If the provided context does NOT contain sufficient information,
state clearly that you cannot answer.
4. Keep your answer accurate, structured, concise, and formatted
with clean Markdown.
```

9.5 REST API Layer

main.py defines the FastAPI application with CORS enabled for all origins, and declares typed Pydantic request/response models (QueryRequest, SourceItem, QueryResponse, UploadResponse) so that FastAPI can automatically validate incoming requests and generate accurate OpenAPI/Swagger documentation. Errors from the document processing, vector store, or RAG pipeline layers are caught and mapped to appropriate HTTP status codes (400 for bad input, 429 for API rate limits, 500 for unexpected/internal errors).

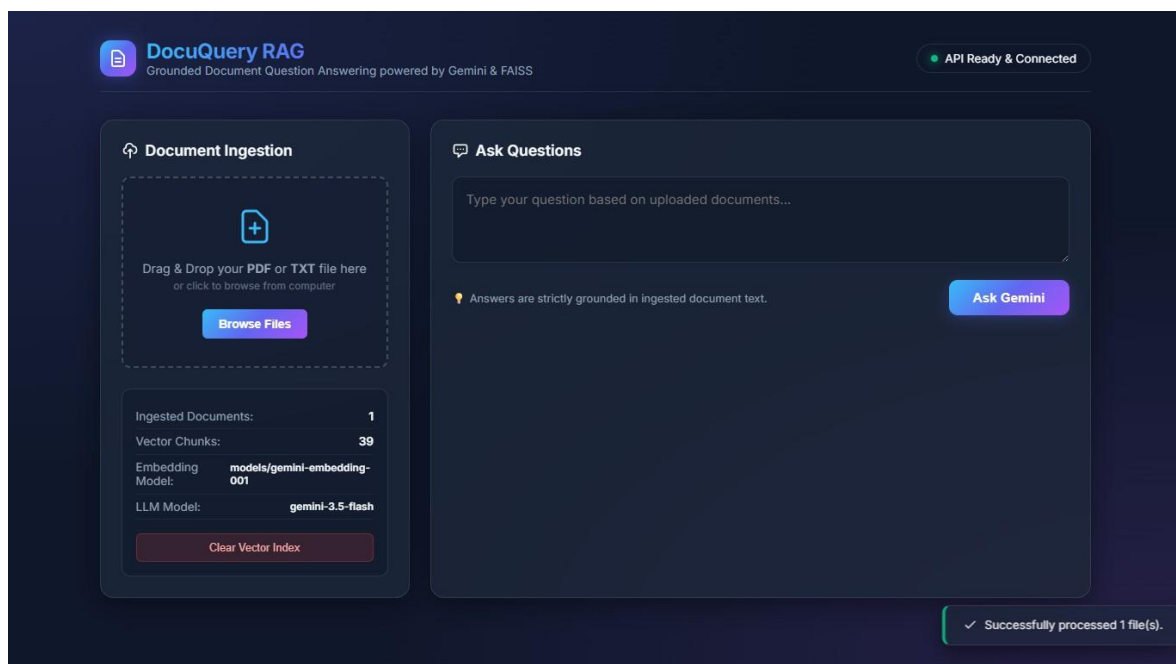
9.6 Front-End Integration

The static/ directory contains a single-page application (index.html, styles.css, app.js) built with a glassmorphic visual design. The JavaScript layer (app.js) uses the Fetch API to communicate with the /upload and /query endpoints, updates the ingestion statistics panel in real time, and renders the grounded answer along with a collapsible list of retrieved context chunks — each showing the source PDF name, page number, and relevance distance score.

10. User Interface & Screenshots

This section presents the actual working application, captured directly from the developed system.

10.1 Home Page — DocuQuery RAG Interface



10.2 Grounded Query Response with Retrieved Sources

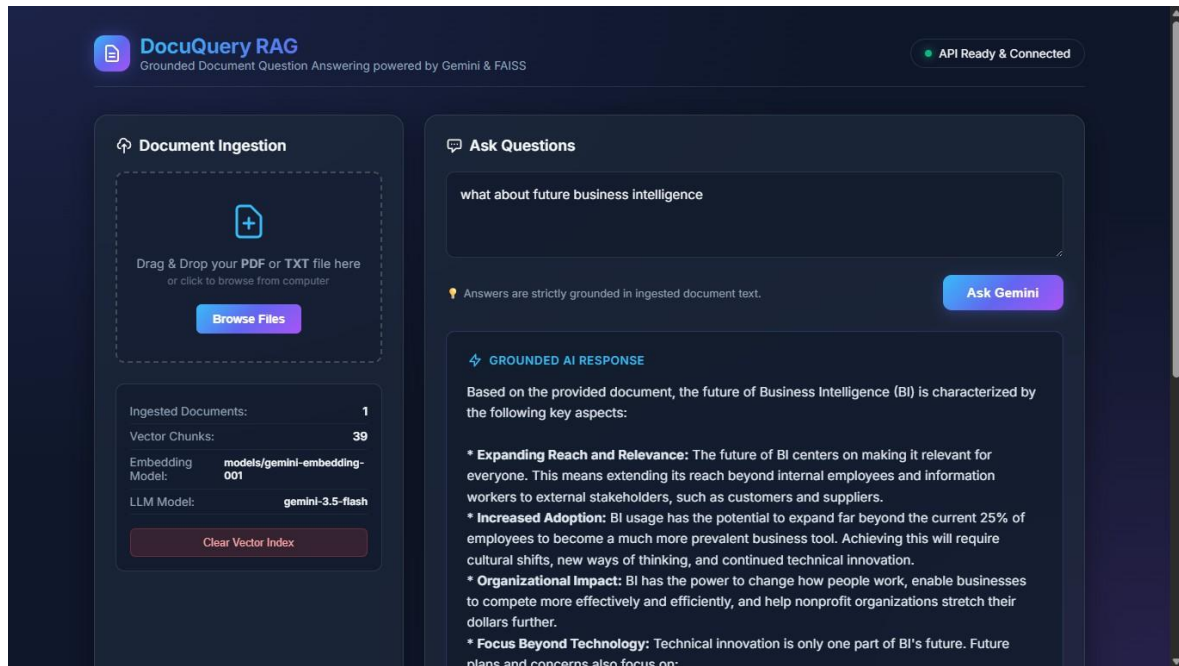


Figure 10.2: A user question ('what about future business intelligence') answered by the Gemini LLM strictly from the ingested document, displayed as a structured, Markdown-formatted 'Grounded AI Response'.

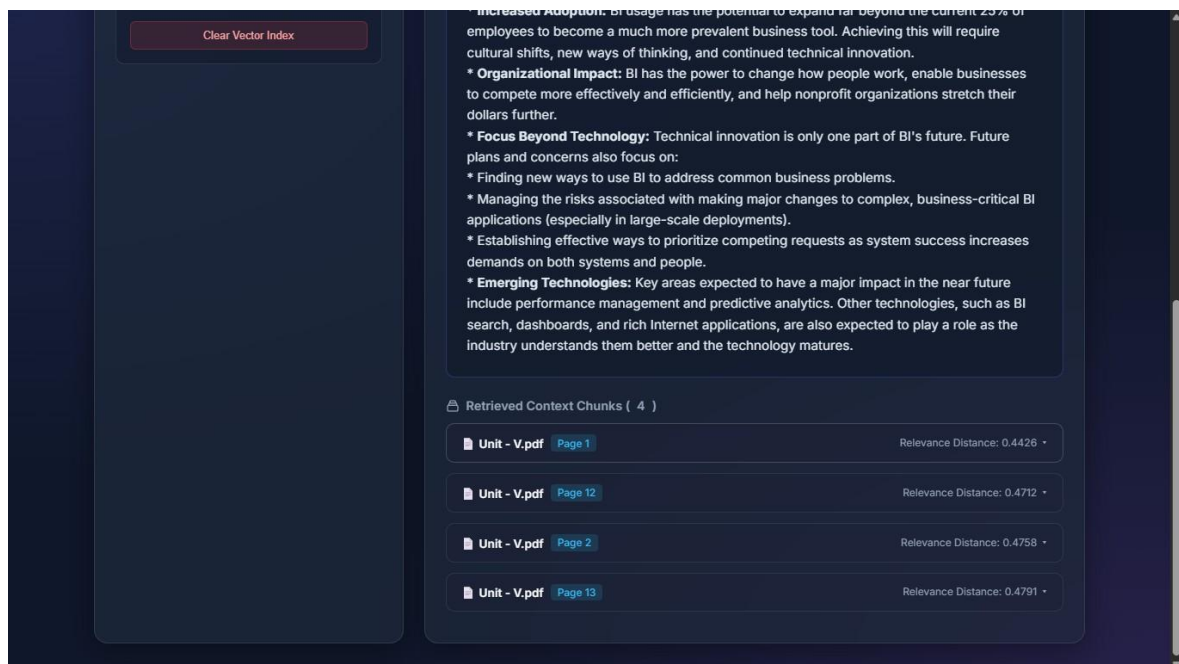


Figure 10.3: The 'Retrieved Context Chunks' section for the same query, listing each matched chunk with its source document (Unit - V.pdf), page number, and relevance distance score — demonstrating full source traceability.

10.3 Backend Server Running (Development Terminal)

```

PS C:\Users\Beast\OneDrive\Desktop\Intermediate-level-project> uvicorn main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [2020] using WatchFiles
INFO: Started server process [11276]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:60401 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:60401 - "GET /static/styles.css HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59308 - "GET /static/app.js HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59308 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:59308 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:62100 - "POST /upload HTTP/1.1" 200 OK
INFO: 127.0.0.1:62100 - "GET /health HTTP/1.1" 200 OK
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
INFO: 127.0.0.1:65023 - "POST /query HTTP/1.1" 200 OK
INFO: 127.0.0.1:54550 - "GET /doc HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60484 - "GET /doc HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60484 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:60484 - "GET /openapi.json HTTP/1.1" 200 OK
  
```

Figure 10.4: VS Code terminal showing the Uvicorn ASGI server running the FastAPI application (`uvicorn main:app --reload`) with live request logs for the web UI, `/health`, `/upload`, `/query`, and Swagger documentation routes.

11. Testing and Results

The project includes an automated test suite, `test_rag.py`, built with **pytest** and FastAPI's **TestClient**. A terminal screenshot of an executed pytest run was not provided at the time of writing this report; the test cases below are therefore described directly from the source code rather than presented as an executed 'results' screenshot, in line with the requirement to avoid claiming unverified results.

11.1 Test Cases Implemented

Test Function	What It Verifies
<code>test_config_defaults</code>	Confirms default configuration values (embedding model, LLM model, chunk size = 1000, chunk overlap = 200, top_k = 4) are loaded correctly from <code>config.py</code>
<code>test_extract_text_from_txt</code>	Confirms plain text is correctly extracted from a <code>.txt</code> file with correct source and page metadata
<code>test_extract_text_from_pdf</code>	Confirms text is correctly extracted from a PDF generated with PyMuPDF, with correct source and page metadata
<code>test_process_document_chunking</code>	Confirms that a long text document is split into multiple chunks, each respecting the configured chunk size limit
<code>test_unsupported_file_extension</code>	Confirms that a <code>DocumentProcessingError</code> is correctly raised when an unsupported file type (e.g., <code>.png</code>) is submitted
<code>test_api_health</code>	Confirms the GET <code>/health</code> endpoint returns HTTP 200 with a 'healthy' status and vector store statistics
<code>test_api_clear</code>	Confirms the POST <code>/clear</code> endpoint returns HTTP 200 and successfully resets the vector index

11.2 Functional / Manual Testing (Verified via Screenshots)

- Document upload: A PDF (Unit - V.pdf) was successfully uploaded and ingested, producing 39 vector chunks from 1 document, as confirmed by the ingestion statistics panel and the 'Successfully processed 1 file(s)' toast notification.
- Grounded querying: A natural-language question was submitted and answered correctly using only the ingested document's content, with the response correctly structured in Markdown with headings and bullet points.
- Source traceability: The retrieved context chunks were displayed with accurate page numbers (Page 1, 2, 12, 13) and relevance distance scores (0.4426–0.4791), confirming the similarity search and metadata pipeline function correctly end-to-end.
- API contract verification: All API endpoints (/ , /health, /upload, /query, /clear) were verified through the auto-generated Swagger UI at /docs, confirming correct request/response schemas as declared in main.py.
- Server stability: The Uvicorn development server logs confirm successful startup, auto-reload functionality, and correct HTTP response codes (200 OK) for all core routes during manual testing.

11.3 API Documentation (Swagger / OpenAPI)

The following screenshots, captured from the interactive Swagger UI at <http://localhost:8000/docs>, verify the structure and correctness of each REST endpoint as implemented in main.py.

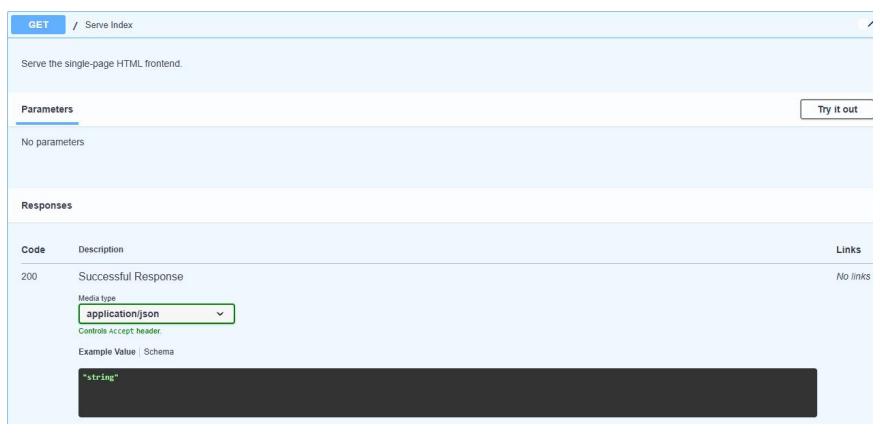


Figure 11.1: GET / — Serves the single-page HTML frontend.

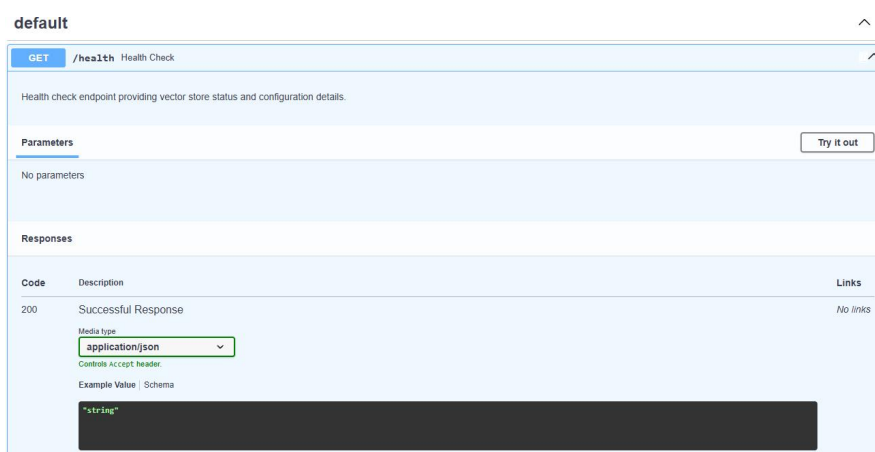


Figure 11.2: GET /health — Health check endpoint providing vector store status and configuration details.

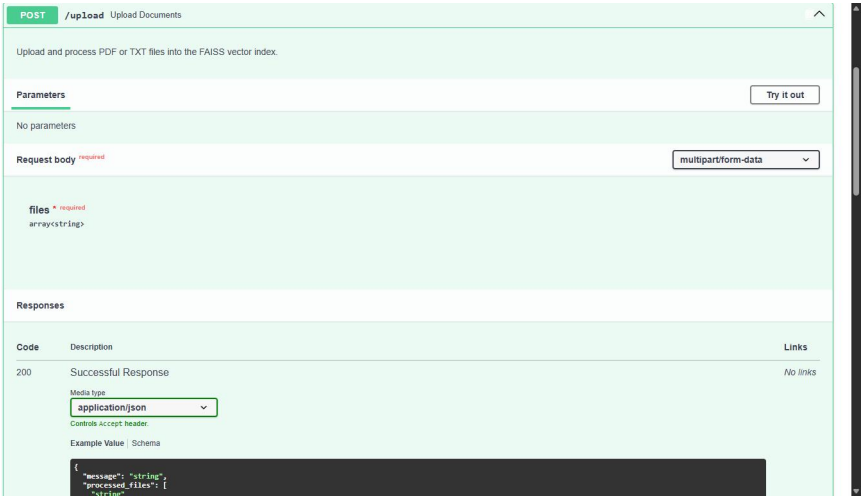


Figure 11.3: `POST /upload` — Accepts `multipart/form-data` file uploads (PDF/TXT) for ingestion into the FAISS vector index.



Figure 11.4: `POST /upload` response schema — returns `message`, `processed_files`, `chunks_added`, and `total_chunks`, with a `422` validation error schema for invalid input.

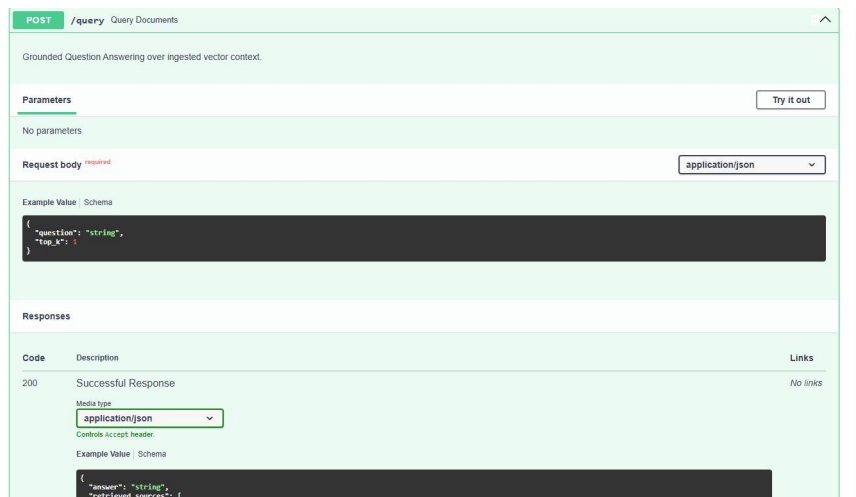


Figure 11.5: POST /query — Accepts a JSON body with 'question' and optional 'top_k', performing grounded question answering over the ingested vector context.

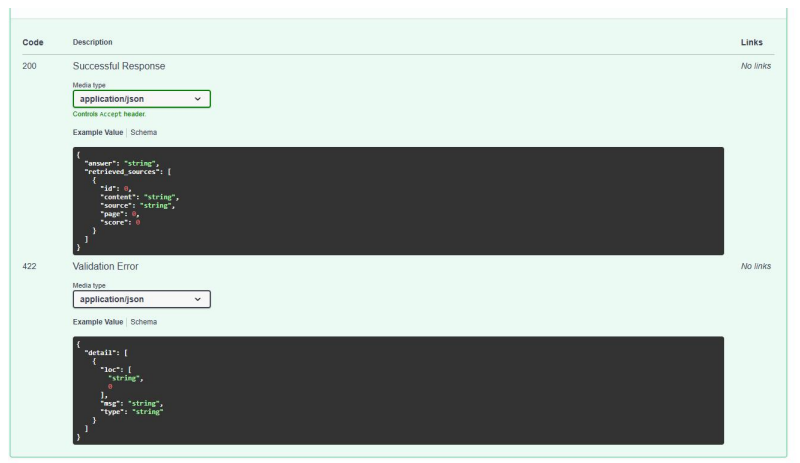


Figure 11.6: POST /query response schema — returns 'answer' and 'retrieved_sources' (each with id, content, source, page, and score).

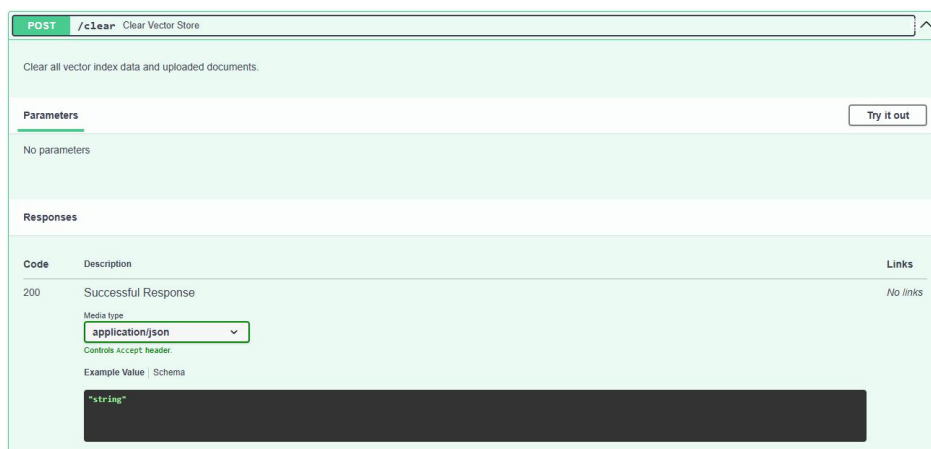


Figure 11.7: POST /clear — Clears all vector index data and uploaded documents, resetting the system to its initial state.

12. Advantages

- Significantly reduces hallucination by forcing the LLM to answer strictly from retrieved document context, with an explicit refusal when context is insufficient.
- Full source transparency: every answer is backed by traceable source document names, page numbers, and relevance scores.
- Modular, maintainable codebase with clear separation of concerns across configuration, document processing, vector storage, RAG logic, and API layers.
- Resilient to Gemini API rate limits through automatic retry with exponential backoff on both the embedding and generation calls.
- Fast local similarity search using FAISS, avoiding the latency and cost of external vector database services for small-to-medium document collections.
- Self-documenting REST API via FastAPI's automatic Swagger/OpenAPI generation, simplifying integration and testing.
- Lightweight, dependency-light front-end (vanilla HTML/CSS/JS) that runs without any separate build step or frontend framework.

13. Limitations

- The FAISS vector index is held entirely in memory and is not persisted to disk; all ingested documents and embeddings are lost when the server restarts.
- The system depends entirely on the external Google Gemini API for both embeddings and answer generation, making it subject to that service's availability, latency, and rate limits (free-tier quota constraints are explicitly handled but still limit throughput).
- Only PDF and TXT file formats are currently supported; formats such as DOCX, HTML, or scanned (image-only) PDFs requiring OCR are not handled.
- There is no user authentication, multi-user session isolation, or per-user document namespace — the vector index is shared globally across all requests to a given server instance.
- No persistent chat history or conversational memory across multiple questions; each query is treated independently.
- Large PDF documents may take noticeably longer to ingest due to sequential per-page text extraction and chunk embedding calls.

14. Future Enhancements

- Persist the FAISS index (and associated metadata) to disk or migrate to a managed vector database so ingested documents survive server restarts.
- Add user authentication and per-user/per-session document namespaces for multi-tenant use.
- Support additional document formats such as DOCX, HTML, Markdown, and OCR-based ingestion for scanned/image-based PDFs.
- Introduce conversational memory so that follow-up questions can reference the context of previous questions in the same session.
- Add batch/asynchronous embedding generation to speed up ingestion of large documents.
- Implement caching for repeated queries to reduce redundant Gemini API calls and cost.
- Add a citation-highlighting feature in the UI that visually highlights the exact retrieved passage within the source document.
- Containerize the application (Docker) and add CI/CD pipeline integration for production deployment.

15. Conclusion

The DocuQuery RAG project, developed during the one-month AI Engineer internship at SHADOWFOX, successfully demonstrates a complete, working Retrieval-Augmented Generation pipeline built on modern, industry-relevant tools: FastAPI for the backend API, PyMuPDF and LangChain for document processing, FAISS for vector similarity search, and the Google Gemini API for embeddings and language generation. The application meets its core objective of providing accurate, source-grounded answers to natural-language questions about user-uploaded documents, with full transparency into which parts of the source material were used to generate each answer.

Beyond the technical deliverable, this internship provided hands-on, practical experience with the core components of modern AI engineering — embeddings, vector databases, prompt engineering for grounded generation, REST API design, and full-stack integration — all of which are directly applicable to real-world AI product development.

16. References

- FastAPI Documentation — <https://fastapi.tiangolo.com/>
- LangChain Documentation — <https://python.langchain.com/>
- FAISS: A Library for Efficient Similarity Search — <https://faiss.ai/>
- Google AI Studio / Gemini API Documentation — <https://aistudio.google.com/>
- PyMuPDF (fitz) Documentation — <https://pymupdf.readthedocs.io/>
- Pydantic Documentation — <https://docs.pydantic.dev/>
- Uvicorn ASGI Server Documentation — <https://www.uvicorn.org/>
- Project source code and README.md (Intermediate-level-project.zip), developed during the SHADOWFOX internship, August–September 2026